

# Detailed Notes on Sorting Algorithms

Merge Sort, Quick Sort, Heap Sort, Counting Sort, and Radix Sort

**Purpose of these notes:** These notes are designed for students who want to understand sorting algorithms from the very beginning in a clear, visual, and coding-friendly way. Each algorithm is explained through definition, stepwise working, important terms, diagrams, complexity analysis, simple algorithm, and Python code.

Prepared for concept clarity, exam preparation, and coding understanding

April 10, 2026

## Contents

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction to Sorting</b>                                   | <b>3</b>  |
| 1.1      | Why do we need sorting? . . . . .                                | 3         |
| 1.2      | Basic ideas students must know before learning sorting . . . . . | 3         |
| 1.3      | Core coding syntax and what it is used for . . . . .             | 4         |
| 1.4      | Main performance measures in sorting . . . . .                   | 4         |
| <b>2</b> | <b>Merge Sort</b>                                                | <b>5</b>  |
| 2.1      | Definition . . . . .                                             | 5         |
| 2.2      | Main idea in one line . . . . .                                  | 5         |
| 2.3      | Important terms in Merge Sort . . . . .                          | 5         |
| 2.4      | How Merge Sort works conceptually . . . . .                      | 5         |
| 2.4.1    | Step 1: Divide the array . . . . .                               | 5         |
| 2.4.2    | Step 2: Merge sorted parts . . . . .                             | 5         |
| 2.5      | Visual flow of Merge Sort . . . . .                              | 6         |
| 2.6      | Detailed merging example . . . . .                               | 6         |
| 2.7      | Why Merge Sort is stable . . . . .                               | 6         |
| 2.8      | Time complexity of Merge Sort . . . . .                          | 6         |
| 2.9      | Space complexity of Merge Sort . . . . .                         | 6         |
| 2.10     | When should we use Merge Sort? . . . . .                         | 7         |
| 2.11     | When is Merge Sort not ideal? . . . . .                          | 7         |
| 2.12     | Simple algorithm of Merge Sort in plain language . . . . .       | 7         |
| 2.13     | Python code for Merge Sort . . . . .                             | 7         |
| 2.14     | Line-by-line understanding of important syntax . . . . .         | 8         |
| <b>3</b> | <b>Quick Sort</b>                                                | <b>8</b>  |
| 3.1      | Definition . . . . .                                             | 8         |
| 3.2      | Main idea in one line . . . . .                                  | 8         |
| 3.3      | Important terms in Quick Sort . . . . .                          | 9         |
| 3.4      | Different ways to choose pivot . . . . .                         | 9         |
| 3.5      | Working example of Quick Sort . . . . .                          | 9         |
| 3.6      | Partition idea visually . . . . .                                | 10        |
| 3.7      | Why Quick Sort is usually fast . . . . .                         | 10        |
| 3.8      | When Quick Sort becomes slow . . . . .                           | 10        |
| 3.9      | Time complexity of Quick Sort . . . . .                          | 11        |
| 3.10     | Space complexity of Quick Sort . . . . .                         | 11        |
| 3.11     | Is Quick Sort stable? . . . . .                                  | 11        |
| 3.12     | When should we use Quick Sort? . . . . .                         | 11        |
| 3.13     | When should we avoid standard Quick Sort? . . . . .              | 11        |
| 3.14     | Simple algorithm of Quick Sort in plain language . . . . .       | 11        |
| 3.15     | Python code for Quick Sort (Lomuto Partition) . . . . .          | 12        |
| 3.16     | Line-by-line understanding of important syntax . . . . .         | 12        |
| 3.17     | One more small example of partitioning . . . . .                 | 12        |
| <b>4</b> | <b>Heap Sort</b>                                                 | <b>13</b> |
| 4.1      | Definition . . . . .                                             | 13        |
| 4.2      | What is a heap? . . . . .                                        | 13        |
| 4.3      | Array representation of heap . . . . .                           | 13        |
| 4.4      | Main idea of Heap Sort . . . . .                                 | 13        |
| 4.5      | Example of Heap Sort . . . . .                                   | 14        |
| 4.5.1    | Step 1: Build max heap . . . . .                                 | 14        |

|           |                                                     |           |
|-----------|-----------------------------------------------------|-----------|
| 4.5.2     | Step 2: Move largest element to end                 | 14        |
| 4.5.3     | Step 3: Repeat                                      | 14        |
| 4.6       | Visual intuition of Heap Sort                       | 14        |
| 4.7       | What is heapify?                                    | 15        |
| 4.8       | Time complexity of Heap Sort                        | 15        |
| 4.9       | Space complexity of Heap Sort                       | 15        |
| 4.10      | Is Heap Sort stable?                                | 15        |
| 4.11      | When should we use Heap Sort?                       | 15        |
| 4.12      | Simple algorithm of Heap Sort in plain language     | 16        |
| 4.13      | Python code for Heap Sort                           | 16        |
| 4.14      | Line-by-line understanding of important syntax      | 16        |
| <b>5</b>  | <b>Non-Comparison Sorts (Conceptual)</b>            | <b>17</b> |
| 5.1       | Counting Sort (Conceptual)                          | 17        |
| 5.1.1     | Definition                                          | 17        |
| 5.1.2     | When is Counting Sort useful?                       | 17        |
| 5.1.3     | Basic idea                                          | 17        |
| 5.1.4     | Step 1: Count frequencies                           | 17        |
| 5.1.5     | Step 2: Reconstruct sorted array                    | 18        |
| 5.1.6     | Visual understanding                                | 18        |
| 5.1.7     | Complexity                                          | 18        |
| 5.1.8     | Stability                                           | 18        |
| 5.1.9     | Simple conceptual algorithm                         | 18        |
| 5.1.10    | Short Python concept code                           | 19        |
| 5.1.11    | Limitations                                         | 19        |
| 5.2       | Radix Sort (Conceptual)                             | 19        |
| 5.2.1     | Definition                                          | 19        |
| 5.2.2     | When is Radix Sort useful?                          | 19        |
| 5.2.3     | Example                                             | 19        |
| 5.2.4     | Key point                                           | 20        |
| 5.2.5     | Complexity                                          | 20        |
| 5.2.6     | Stability                                           | 20        |
| 5.2.7     | Simple conceptual algorithm                         | 20        |
| 5.2.8     | Short Python concept code                           | 21        |
| 5.2.9     | Limitations                                         | 21        |
| <b>6</b>  | <b>Complete Comparison Table</b>                    | <b>21</b> |
| <b>7</b>  | <b>How to Decide Which Sorting Algorithm to Use</b> | <b>22</b> |
| <b>8</b>  | <b>Common Mistakes Students Make</b>                | <b>22</b> |
| <b>9</b>  | <b>Quick Revision Summary</b>                       | <b>23</b> |
| <b>10</b> | <b>Practice Questions for Students</b>              | <b>23</b> |
| <b>11</b> | <b>Conclusion</b>                                   | <b>23</b> |

# 1 Introduction to Sorting

## What is sorting?

Sorting is the process of arranging data in a specific order, usually **ascending** or **descending**. For example, if the numbers are

8, 3, 1, 6, 2

then after sorting in ascending order, the result becomes

1, 2, 3, 6, 8

## 1.1 Why do we need sorting?

Sorting is one of the most important operations in computer science because it helps in:

- faster searching,
- easier data analysis,
- better report generation,
- efficient use of algorithms such as binary search,
- arranging records like marks, salary, names, or dates.

## 1.2 Basic ideas students must know before learning sorting

| Term                     | Meaning                                                                                                                          |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Comparison based sorting | Sorting is done by comparing two elements, such as checking whether $a < b$ or $a > b$ .                                         |
| Non-comparison sorting   | Sorting is not done mainly by direct comparisons. It uses some property of the input, such as digit position or frequency count. |
| Stable sorting           | If two equal values appear in the input, their original relative order remains unchanged after sorting.                          |
| Unstable sorting         | Equal elements may change their original relative order.                                                                         |
| In-place sorting         | Sorting is done using very little extra memory.                                                                                  |
| Out-of-place sorting     | Additional memory is needed to complete sorting.                                                                                 |
| Divide and Conquer       | A big problem is divided into smaller subproblems, solved separately, then combined.                                             |
| Recursion                | A function calls itself to solve a smaller version of the same problem.                                                          |
| Pivot                    | A selected element used by Quick Sort to divide the array into two parts.                                                        |
| Heap                     | A special complete binary tree used in Heap Sort.                                                                                |
| Merging                  | Combining two already sorted lists into one sorted list.                                                                         |
| Partitioning             | Rearranging an array around a pivot such that smaller values go left and larger values go right.                                 |

### 1.3 Core coding syntax and what it is used for

Students often know the algorithm idea but do not understand *which coding syntax performs which action*. The table below connects the concept with Python syntax.

| Coding action                    | Python syntax                            | Use in sorting                                                                                     |
|----------------------------------|------------------------------------------|----------------------------------------------------------------------------------------------------|
| Condition checking               | <code>if ... elif ... else</code>        | Used to compare values and decide what to do next. Example: choose smaller element during merging. |
| Repeated step-by-step movement   | <code>for</code> loop                    | Used when we need to move index by index through the array.                                        |
| Repeat until a condition changes | <code>while</code> loop                  | Useful in partitioning, heapify, or merging while elements remain.                                 |
| Function creation                | <code>def function_name():</code>        | Used to write reusable logic such as <code>merge_sort()</code> or <code>heapify()</code> .         |
| Recursive call                   | <code>function_name(smaller_part)</code> | Used in divide-and-conquer algorithms like Merge Sort and Quick Sort.                              |
| Swapping values                  | <code>a[i], a[j] = a[j], a[i]</code>     | Used in Quick Sort and Heap Sort to exchange positions.                                            |
| List slicing                     | <code>arr[:mid], arr[mid:]</code>        | Used to divide the array into left and right halves in Merge Sort.                                 |
| Index access                     | <code>arr[i]</code>                      | Used to read a specific element.                                                                   |
| Length of array                  | <code>len(arr)</code>                    | Used to know size and stopping condition.                                                          |
| Temporary array                  | <code>temp = []</code>                   | Used to store merged results or counts.                                                            |

### 1.4 Main performance measures in sorting

#### Time Complexity

Time complexity tells us how the running time grows when the input size grows.

#### Space Complexity

Space complexity tells us how much extra memory an algorithm requires.

The most common notations are:

$$O(1), \quad O(\log n), \quad O(n), \quad O(n \log n), \quad O(n^2)$$

#### Important

In sorting, we usually discuss:

- **Best case**
- **Average case**
- **Worst case**

because some algorithms behave differently for different input arrangements.

## 2 Merge Sort

### 2.1 Definition

#### Merge Sort

Merge Sort is a **divide-and-conquer** sorting algorithm. It divides the array into two halves, recursively sorts each half, and then merges the two sorted halves into one final sorted array.

### 2.2 Main idea in one line

Divide  $\rightarrow$  Sort smaller parts  $\rightarrow$  Merge

### 2.3 Important terms in Merge Sort

- **Divide:** Split the array into two halves.
- **Conquer:** Sort each half recursively.
- **Merge:** Combine two sorted halves into a single sorted list.
- **Stable:** Yes, Merge Sort is stable.
- **Extra space:** Merge Sort needs additional memory.

### 2.4 How Merge Sort works conceptually

Suppose we have:

[38, 27, 43, 3, 9, 82, 10]

#### 2.4.1 Step 1: Divide the array

[38, 27, 43, 3, 9, 82, 10]

Split into:

[38, 27, 43]      [3, 9, 82, 10]

Further split:

[38]   [27, 43]                      [3, 9]   [82, 10]

Further split:

[27]   [43]            [3]   [9]            [82]   [10]

Now every part has one element, so each part is already sorted.

#### 2.4.2 Step 2: Merge sorted parts

[27] + [43]  $\rightarrow$  [27, 43]

[38] + [27, 43]  $\rightarrow$  [27, 38, 43]

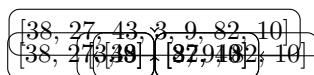
[3] + [9]  $\rightarrow$  [3, 9]

[82] + [10]  $\rightarrow$  [10, 82]

[3, 9] + [10, 82]  $\rightarrow$  [3, 9, 10, 82]

[27, 38, 43] + [3, 9, 10, 82]  $\rightarrow$  [3, 9, 10, 27, 38, 43, 82]

## 2.5 Visual flow of Merge Sort



## 2.6 Detailed merging example

Take two sorted arrays:

$$L = [27, 38, 43], \quad R = [3, 9, 10, 82]$$

Now compare the front elements one by one.

| Step | Left front | Right front | Action                 | Output so far              |
|------|------------|-------------|------------------------|----------------------------|
| 1    | 27         | 3           | 3 is smaller, take 3   | [3]                        |
| 2    | 27         | 9           | 9 is smaller, take 9   | [3, 9]                     |
| 3    | 27         | 10          | 10 is smaller, take 10 | [3, 9, 10]                 |
| 4    | 27         | 82          | 27 is smaller, take 27 | [3, 9, 10, 27]             |
| 5    | 38         | 82          | 38 is smaller, take 38 | [3, 9, 10, 27, 38]         |
| 6    | 43         | 82          | 43 is smaller, take 43 | [3, 9, 10, 27, 38, 43]     |
| 7    | empty      | 82          | Add remaining element  | [3, 9, 10, 27, 38, 43, 82] |

## 2.7 Why Merge Sort is stable

Suppose we have repeated values:

$$[(5, A), (3, B), (5, C)]$$

If two equal values are compared, Merge Sort can keep the earlier one first during merging. So  $(5, A)$  remains before  $(5, C)$ . That is why Merge Sort is called **stable**.

## 2.8 Time complexity of Merge Sort

At each level:

- the array is divided into two parts,
- all elements are merged back in linear time.

There are about  $\log_2 n$  levels, and at each level the work is about  $n$ .  
Therefore,

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

So:

$$T(n) = O(n \log n)$$

| Case         | Time Complexity |
|--------------|-----------------|
| Best case    | $O(n \log n)$   |
| Average case | $O(n \log n)$   |
| Worst case   | $O(n \log n)$   |

## 2.9 Space complexity of Merge Sort

Merge Sort uses an extra temporary array during merging. Therefore:

$$\text{Space Complexity} = O(n)$$

## 2.10 When should we use Merge Sort?

- When stable sorting is needed.
- When guaranteed  $O(n \log n)$  performance is required.
- When linked lists or external sorting are involved.

## 2.11 When is Merge Sort not ideal?

- When memory is very limited.
- When in-place sorting is preferred.

## 2.12 Simple algorithm of Merge Sort in plain language

### Algorithm: Merge Sort in simple words

1. If the array has only one element, it is already sorted.
2. Divide the array into two halves.
3. Sort the left half using Merge Sort.
4. Sort the right half using Merge Sort.
5. Merge the two sorted halves by repeatedly taking the smaller front element.
6. Return the merged sorted array.

## 2.13 Python code for Merge Sort

Listing 1: Merge Sort in Python

```
def merge_sort(arr):  
    # Base case: if array has 0 or 1 element, it is already sorted  
    if len(arr) <= 1:  
        return arr  
  
    # Divide step  
    mid = len(arr) // 2  
    left_half = arr[:mid]  
    right_half = arr[mid:]  
  
    # Recursively sort both halves  
    left_sorted = merge_sort(left_half)  
    right_sorted = merge_sort(right_half)  
  
    # Merge the sorted halves  
    return merge(left_sorted, right_sorted)  
  
def merge(left, right):  
    result = []  
    i = 0 # pointer for left array  
    j = 0 # pointer for right array  
  
    # Compare elements and append smaller one  
    while i < len(left) and j < len(right):
```



```
    if left[i] <= right[j]: # <= keeps it stable
        result.append(left[i])
        i += 1
    else:
        result.append(right[j])
        j += 1

# Add remaining elements if any
while i < len(left):
    result.append(left[i])
    i += 1

while j < len(right):
    result.append(right[j])
    j += 1

return result

# Example
arr = [38, 27, 43, 3, 9, 82, 10]
print("Original:", arr)
print("Sorted :", merge_sort(arr))
```

## 2.14 Line-by-line understanding of important syntax

- `if len(arr) <= 1`: checks the stopping condition.
- `mid = len(arr) // 2` finds the middle index.
- `arr[:mid]` and `arr[mid:]` divide the array.
- `merge_sort(left_half)` is recursive calling.
- `while i < len(left) and j < len(right)`: means continue until one list is finished.
- `if left[i] <= right[j]`: compares two current front elements.
- `result.append(...)` stores output step by step.

## 3 Quick Sort

### 3.1 Definition

#### Quick Sort

Quick Sort is a divide-and-conquer sorting algorithm that selects one element as a **pivot**, partitions the array around the pivot, and then recursively sorts the left and right parts.

### 3.2 Main idea in one line

Choose pivot → Partition → Recursively sort left and right

### 3.3 Important terms in Quick Sort

- **Pivot:** the chosen element around which partitioning happens.
- **Partitioning:** placing smaller elements on the left and larger elements on the right.
- **Recursive subarrays:** after partitioning, sort left and right parts separately.
- **In-place behavior:** Quick Sort can be implemented mostly in-place.

### 3.4 Different ways to choose pivot

A pivot can be chosen as:

- first element,
- last element,
- middle element,
- random element,
- median-of-three.

#### Important

The choice of pivot affects performance a lot. A poor pivot can make Quick Sort slow.

### 3.5 Working example of Quick Sort

Take the array:

[29, 10, 14, 37, 13]

Assume we choose the **last element** as pivot:

Pivot = 13

Now compare every element with pivot 13:

29, 10, 14, 37, 13

Elements smaller than 13:

10

Elements greater than or equal to 13:

29, 14, 37

After partitioning, pivot reaches its correct position:

[10, 13, 29, 14, 37]

Now recursively sort:

[10]      [29, 14, 37]

For right part, choose pivot 37:

[29, 14, 37]

Both 29 and 14 are smaller than 37, so pivot remains at end:

[29, 14, 37]

Now sort left part of that:

[29, 14]

Choose pivot 14:

[14, 29]

Final sorted array:

[10, 13, 14, 29, 37]

### 3.6 Partition idea visually

Suppose:

[8, 3, 1, 7, 0, 10, 2]

Choose pivot = 2.

Now scan the array from left to right:

- Values smaller than 2 move toward the left side.
- Values greater than or equal to 2 stay on the right side.

Finally, pivot is placed in between:

[1, 0, 2, 7, 3, 10, 8]

So now:

$\underbrace{[1, 0]}_{\text{smaller than pivot}} \quad [2] \quad \underbrace{[7, 3, 10, 8]}_{\text{greater than pivot}}$

### 3.7 Why Quick Sort is usually fast

Quick Sort has very good practical performance because:

- partitioning is simple,
- memory usage is low,
- cache performance is good,
- on average it divides the array fairly well.

### 3.8 When Quick Sort becomes slow

Quick Sort becomes slow when partitioning is highly unbalanced.

Example:

[1, 2, 3, 4, 5]

If the last element is always chosen as pivot, then one side gets almost all elements and the other gets none.

This leads to:

$$T(n) = T(n - 1) + O(n)$$

which becomes:

$$O(n^2)$$

### 3.9 Time complexity of Quick Sort

| Case         | Time Complexity |
|--------------|-----------------|
| Best case    | $O(n \log n)$   |
| Average case | $O(n \log n)$   |
| Worst case   | $O(n^2)$        |

### 3.10 Space complexity of Quick Sort

Quick Sort does not need an extra large array like Merge Sort. It mainly needs recursion stack space.

$$\text{Average space} = O(\log n)$$

$$\text{Worst space} = O(n)$$

### 3.11 Is Quick Sort stable?

Standard Quick Sort is generally **not stable**. Equal elements may change their original relative positions.

### 3.12 When should we use Quick Sort?

- When average performance matters.
- When extra memory should be small.
- When in-place style sorting is preferred.

### 3.13 When should we avoid standard Quick Sort?

- When guaranteed worst-case performance is important.
- When stability is required.

### 3.14 Simple algorithm of Quick Sort in plain language

#### Algorithm: Quick Sort in simple words

1. Choose one element as pivot.
2. Rearrange the array so that smaller elements come before pivot and larger elements come after pivot.
3. Now pivot is in its correct sorted position.
4. Apply the same process to the left part.
5. Apply the same process to the right part.
6. Continue until each part has zero or one element.

### 3.15 Python code for Quick Sort (Lomuto Partition)

Listing 2: Quick Sort in Python using Lomuto Partition Scheme

```
def quick_sort(arr, low, high):
    if low < high:
        # Partition the array and get pivot index
        pi = partition(arr, low, high)

        # Recursively sort left and right subarrays
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)

def partition(arr, low, high):
    pivot = arr[high] # choose last element as pivot
    i = low - 1 # index of smaller element

    for j in range(low, high):
        if arr[j] < pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i] # swap

    # Put pivot in correct position
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

# Example
arr = [29, 10, 14, 37, 13]
print("Original:", arr)
quick_sort(arr, 0, len(arr) - 1)
print("Sorted :", arr)
```

### 3.16 Line-by-line understanding of important syntax

- `if low < high:` checks whether more than one element exists.
- `pivot = arr[high]` selects the last element as pivot.
- `for j in range(low, high):` scans all elements before pivot.
- `if arr[j] < pivot:` checks whether current element belongs to left side.
- `arr[i], arr[j] = arr[j], arr[i]` swaps elements.
- `return i + 1` gives the final correct position of pivot.

### 3.17 One more small example of partitioning

Array:

[4, 2, 7, 1, 3]

Pivot = 3

| Step | Current element | Compare with pivot | Action                                    |
|------|-----------------|--------------------|-------------------------------------------|
| 1    | 4               | $4 < 3$ ? No       | Keep on right side                        |
| 2    | 2               | $2 < 3$ ? Yes      | Move toward left side                     |
| 3    | 7               | $7 < 3$ ? No       | Keep on right side                        |
| 4    | 1               | $1 < 3$ ? Yes      | Move toward left side                     |
| End  | Pivot 3         | —                  | Place pivot between left and right groups |

Final arrangement:

[2, 1, 3, 4, 7]

## 4 Heap Sort

### 4.1 Definition

#### Heap Sort

Heap Sort is a comparison-based sorting algorithm that uses a **heap data structure**, usually a **max heap**, to sort elements.

### 4.2 What is a heap?

A heap is a special **complete binary tree**.

- In a **max heap**, every parent is greater than or equal to its children.
- In a **min heap**, every parent is smaller than or equal to its children.

Heap Sort for ascending order usually uses a **max heap**.

### 4.3 Array representation of heap

If an element is at index  $i$ , then:

$$\text{Left child} = 2i + 1$$

$$\text{Right child} = 2i + 2$$

$$\text{Parent} = \left\lfloor \frac{i - 1}{2} \right\rfloor$$

### 4.4 Main idea of Heap Sort

1. Build a max heap from the array.
2. The largest element is now at the root.
3. Swap the root with the last element.
4. Reduce heap size by one.
5. Heapify again to restore max heap.
6. Repeat until all elements are sorted.

## 4.5 Example of Heap Sort

Take:

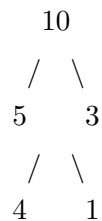
[4, 10, 3, 5, 1]

### 4.5.1 Step 1: Build max heap

After building max heap:

[10, 5, 3, 4, 1]

Its tree view:



### 4.5.2 Step 2: Move largest element to end

Swap first and last:

[1, 5, 3, 4, 10]

Now 10 is fixed at final position.

Heapify remaining part:

[5, 4, 3, 1, 10]

### 4.5.3 Step 3: Repeat

Swap root with last element of current heap:

[1, 4, 3, 5, 10]

Heapify:

[4, 1, 3, 5, 10]

Again swap root with last of current heap:

[3, 1, 4, 5, 10]

Again:

[1, 3, 4, 5, 10]

Final sorted array:

[1, 3, 4, 5, 10]

## 4.6 Visual intuition of Heap Sort

In a max heap:

- the largest element always comes to the top,
- so each round we remove the largest and put it at the end,
- then we rebuild the heap property for the remaining elements.

## 4.7 What is heapify?

### Heapify

Heapify is the process of adjusting a tree so that it satisfies the heap property.

Suppose we have:

[4, 10, 3]

Here 10 is larger than parent 4, so heap property is broken.

After heapify:

[10, 4, 3]

## 4.8 Time complexity of Heap Sort

Heap Sort has two major phases:

- Build heap:  $O(n)$
- Repeated heapify after each extraction:  $O(n \log n)$

So total:

$$O(n \log n)$$

| Case         | Time Complexity |
|--------------|-----------------|
| Best case    | $O(n \log n)$   |
| Average case | $O(n \log n)$   |
| Worst case   | $O(n \log n)$   |

## 4.9 Space complexity of Heap Sort

Heap Sort is considered an **in-place** algorithm because it sorts inside the same array with only very small extra memory.

$$\text{Space Complexity} = O(1)$$

## 4.10 Is Heap Sort stable?

No. Standard Heap Sort is **not stable**.

## 4.11 When should we use Heap Sort?

- When guaranteed  $O(n \log n)$  time is needed.
- When extra memory should remain very small.
- When stability is not required.



## 4.12 Simple algorithm of Heap Sort in plain language

### Algorithm: Heap Sort in simple words

1. Convert the array into a max heap.
2. Swap the first element (largest) with the last unsorted element.
3. Reduce the heap size by one.
4. Restore max heap property using heapify.
5. Repeat until only one element remains.

## 4.13 Python code for Heap Sort

Listing 3: Heap Sort in Python

```
def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    # Check left child
    if left < n and arr[left] > arr[largest]:
        largest = left

    # Check right child
    if right < n and arr[right] > arr[largest]:
        largest = right

    # If root is not largest, swap and continue heapifying
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def heap_sort(arr):
    n = len(arr)

    # Build max heap
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    # Extract elements one by one
    for i in range(n - 1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i] # Move current largest to end
        heapify(arr, i, 0) # Restore heap on reduced heap

# Example
arr = [4, 10, 3, 5, 1]
print("Original:", arr)
heap_sort(arr)
print("Sorted :", arr)
```

## 4.14 Line-by-line understanding of important syntax

- `left = 2 * i + 1` computes left child index.

- `right = 2 * i + 2` computes right child index.
- `if left < n and arr[left] > arr[largest]`: checks whether left child exists and is larger.
- `if largest != i`: means current root is not largest, so swap is needed.
- `for i in range(n // 2 - 1, -1, -1)`: builds heap from bottom to top.
- `for i in range(n - 1, 0, -1)`: repeatedly places largest element at the end.

## 5 Non-Comparison Sorts (Conceptual)

### Non-comparison sorting

These algorithms do not mainly rely on comparing one element with another. Instead, they use information such as **count**, **digit position**, or **range of values**.

### Important

Non-comparison sorts can be faster than  $O(n \log n)$  in special situations, but they are not suitable for all kinds of data.

### 5.1 Counting Sort (Conceptual)

#### 5.1.1 Definition

Counting Sort works by counting how many times each value appears, then using that count information to place elements in sorted order.

#### 5.1.2 When is Counting Sort useful?

Counting Sort is useful when:

- values are integers,
- the range of values is not very large,
- for example, marks from 0 to 100, ages from 1 to 120, small category IDs.

#### 5.1.3 Basic idea

For input:

[4, 2, 2, 8, 3, 3, 1]

Possible values are from 1 to 8. Create a count array of size 9 (or 8+1 if indexing from 0).

#### 5.1.4 Step 1: Count frequencies

Count of 1 = 1

Count of 2 = 2

Count of 3 = 2

Count of 4 = 1

Count of 8 = 1

Count array:

[0, 1, 2, 2, 1, 0, 0, 0, 1]

### 5.1.5 Step 2: Reconstruct sorted array

Write 1 once, 2 twice, 3 twice, 4 once, 8 once:

[1, 2, 2, 3, 3, 4, 8]

### 5.1.6 Visual understanding

Input:

[4, 2, 2, 8, 3, 3, 1]

Frequency table:

| Value | Frequency |
|-------|-----------|
| 1     | 1         |
| 2     | 2         |
| 3     | 2         |
| 4     | 1         |
| 5     | 0         |
| 6     | 0         |
| 7     | 0         |
| 8     | 1         |

Sorted output from frequency:

1, 2, 2, 3, 3, 4, 8

### 5.1.7 Complexity

If  $n$  is number of elements and  $k$  is range of values, then:

$$\text{Time Complexity} = O(n + k)$$

$$\text{Space Complexity} = O(n + k) \text{ or } O(k) \text{ depending on implementation}$$

### 5.1.8 Stability

Counting Sort can be **stable** if implemented carefully using cumulative counts and backward placement.

### 5.1.9 Simple conceptual algorithm

#### Counting Sort in simple words

1. Find the maximum value.
2. Create a count array from 0 to maximum value.
3. Count how many times each value appears.
4. Read the count array from small to large value.
5. Write each value according to its frequency.

### 5.1.10 Short Python concept code

Listing 4: Counting Sort Concept in Python

```
def counting_sort_concept(arr):
    max_val = max(arr)
    count = [0] * (max_val + 1)

    for num in arr:
        count[num] += 1

    result = []
    for value in range(len(count)):
        for _ in range(count[value]):
            result.append(value)

    return result

arr = [4, 2, 2, 8, 3, 3, 1]
print(counting_sort_concept(arr))
```

### 5.1.11 Limitations

- Not suitable when the value range is huge.
- Not suitable for general real numbers or strings directly.

## 5.2 Radix Sort (Conceptual)

### 5.2.1 Definition

Radix Sort sorts numbers digit by digit. It usually starts from the least significant digit (LSD) and moves toward the most significant digit (MSD), while using a stable sort at each step.

### 5.2.2 When is Radix Sort useful?

Radix Sort is useful when:

- data are integers or strings of fixed structure,
- digits or characters can be processed position by position,
- stable intermediate sorting is possible.

### 5.2.3 Example

Take:

[170, 45, 75, 90, 802, 24, 2, 66]

We sort digit by digit.

**Pass 1: Sort by unit digit**    Unit digits:

170 → 0,   45 → 5,   75 → 5,   90 → 0,   802 → 2,   24 → 4,   2 → 2,   66 → 6

After sorting by unit digit:

[170, 90, 802, 2, 24, 45, 75, 66]

**Pass 2: Sort by tens digit** Tens digits:

$170 \rightarrow 7, \quad 90 \rightarrow 9, \quad 802 \rightarrow 0, \quad 2 \rightarrow 0, \quad 24 \rightarrow 2, \quad 45 \rightarrow 4, \quad 75 \rightarrow 7, \quad 66 \rightarrow 6$

After sorting by tens digit:

[802, 2, 24, 45, 66, 170, 75, 90]

**Pass 3: Sort by hundreds digit** Hundreds digits:

$802 \rightarrow 8, \quad 2 \rightarrow 0, \quad 24 \rightarrow 0, \quad 45 \rightarrow 0, \quad 66 \rightarrow 0, \quad 170 \rightarrow 1, \quad 75 \rightarrow 0, \quad 90 \rightarrow 0$

Final sorted array:

[2, 24, 45, 66, 75, 90, 170, 802]

#### 5.2.4 Key point

Radix Sort itself does not directly compare entire numbers. It sorts based on digit positions.

#### 5.2.5 Complexity

If:

- $n$  = number of elements,
- $d$  = number of digits,
- $k$  = range of each digit (for decimal system,  $k = 10$ ),

then:

$$\text{Time Complexity} = O(d(n + k))$$

For decimal digits with fixed small base:

$$\text{Time Complexity} \approx O(dn)$$

$$\text{Space Complexity} = O(n + k)$$

#### 5.2.6 Stability

Radix Sort requires the inner digit-wise sorting method to be **stable**. That is why Counting Sort is often used inside Radix Sort.

#### 5.2.7 Simple conceptual algorithm

##### Radix Sort in simple words

1. Find the largest number to know how many digit positions exist.
2. Sort the numbers by the unit digit.
3. Sort again by the tens digit.
4. Sort again by the hundreds digit.
5. Continue until the highest digit position is processed.

### 5.2.8 Short Python concept code

Listing 5: Radix Sort Concept in Python

```
def counting_sort_by_digit(arr, exp):
    n = len(arr)
    output = [0] * n
    count = [0] * 10

    # Count occurrences of current digit
    for num in arr:
        index = (num // exp) % 10
        count[index] += 1

    # Convert count into position information
    for i in range(1, 10):
        count[i] += count[i - 1]

    # Build output array in reverse to keep it stable
    for i in range(n - 1, -1, -1):
        index = (arr[i] // exp) % 10
        output[count[index] - 1] = arr[i]
        count[index] -= 1

    # Copy output back
    for i in range(n):
        arr[i] = output[i]

def radix_sort_concept(arr):
    max_val = max(arr)
    exp = 1

    while max_val // exp > 0:
        counting_sort_by_digit(arr, exp)
        exp *= 10

    return arr

arr = [170, 45, 75, 90, 802, 24, 2, 66]
print(radix_sort_concept(arr))
```

### 5.2.9 Limitations

- Best suited for integers or fixed-format data.
- Not a general-purpose replacement for all sorting problems.

## 6 Complete Comparison Table

| Algorithm | Best Time | Average Time | Worst Time | Space | Stable? | Main Idea |
|-----------|-----------|--------------|------------|-------|---------|-----------|
|-----------|-----------|--------------|------------|-------|---------|-----------|

|               |               |               |               |                     |                                 |                                      |
|---------------|---------------|---------------|---------------|---------------------|---------------------------------|--------------------------------------|
| Merge Sort    | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(n)$              | Yes                             | Divide, recursively sort, then merge |
| Quick Sort    | $O(n \log n)$ | $O(n \log n)$ | $O(n^2)$      | $O(\log n)$ average | No                              | Partition around pivot               |
| Heap Sort     | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(1)$              | No                              | Build max heap and extract largest   |
| Counting Sort | $O(n + k)$    | $O(n + k)$    | $O(n + k)$    | $O(n + k)$          | Yes (if implemented properly)   | Count frequency of each value        |
| Radix Sort    | $O(d(n + k))$ | $O(d(n + k))$ | $O(d(n + k))$ | $O(n + k)$          | Yes (if stable inner sort used) | Sort digit by digit                  |

## 7 How to Decide Which Sorting Algorithm to Use

| Situation                                    | Recommended sorting idea              |
|----------------------------------------------|---------------------------------------|
| Need stable sorting                          | Merge Sort, Counting Sort, Radix Sort |
| Need low extra memory                        | Heap Sort, Quick Sort                 |
| Need guaranteed $O(n \log n)$ time           | Merge Sort or Heap Sort               |
| Need very fast average practical performance | Quick Sort                            |
| Values are small integers with limited range | Counting Sort                         |
| Numbers have fixed number of digits          | Radix Sort                            |

## 8 Common Mistakes Students Make

### Common mistakes

1. Confusing **stable** and **in-place**.
2. Thinking Quick Sort is always  $O(n \log n)$  even in worst case.
3. Forgetting that Merge Sort needs extra memory.
4. Forgetting child index formulas in heap.
5. Using Counting Sort when the value range is extremely large.
6. Forgetting that Radix Sort needs stable digit-wise sorting.
7. Not understanding recursion stopping condition.

## 9 Quick Revision Summary

### One-page revision summary

- **Merge Sort:** Divide array into halves, sort recursively, merge sorted halves. Stable. Time  $O(n \log n)$ . Space  $O(n)$ .
- **Quick Sort:** Choose pivot, partition around pivot, sort left and right parts recursively. Average  $O(n \log n)$ , worst  $O(n^2)$ . Usually in-place.
- **Heap Sort:** Build max heap, repeatedly place largest element at end. Time  $O(n \log n)$ . Space  $O(1)$ . Not stable.
- **Counting Sort:** Count frequencies of values. Best for small integer range. Time  $O(n + k)$ .
- **Radix Sort:** Sort digit by digit using stable intermediate sort. Good for integers with fixed number of digits.

## 10 Practice Questions for Students

1. What is the difference between stable and unstable sorting? Give examples.
2. Why is Merge Sort called a divide-and-conquer algorithm?
3. Why can Quick Sort become  $O(n^2)$  in the worst case?
4. Why does Heap Sort use a max heap for ascending order?
5. What is the role of pivot in Quick Sort?
6. Why is Counting Sort not suitable for very large ranges?
7. Why must the inner sorting method of Radix Sort be stable?
8. Compare Merge Sort and Heap Sort in terms of memory and stability.
9. Write the child index formulas used in heap representation by array.
10. Sort the following using Merge Sort by showing all major steps:

[12, 11, 13, 5, 6, 7]

## 11 Conclusion

Sorting is a fundamental topic in data structures and algorithms. Understanding only the final code is not enough. A student must know:

- the main idea behind the algorithm,
- why each step is performed,
- what the important terms mean,
- how time and space complexity are derived,
- which coding syntax is used for each action,



- and where a particular algorithm is useful in practice.

If these concepts are clear, then writing code becomes much easier and more meaningful.

**End of Notes**